

التهيئة والتعريف بـ Git وGitHub

01

ورقة عمل

Workshop 1 · Setup & Orientation — Git vs. GitHub

المدة / Duration	المستوى / Level	المتطلبات / Prerequisites	المخرجات / Deliverable	المهام / Tasks	الدرجة / Points
90 دقيقة	مبتدئ — لا يشترط خبرة سابقة	جهاز حاسوب + اتصال إنترنت	حساب GitHub + Git مثبت وقُعدّ	6 مهام	100 نقطة

اسم الطالب / Student	اسم المستخدم / GitHub username	المجموعة / Group	التاريخ / Date

كيف تعمل هذه الورقة؟ How this worksheet works

هذه الورقة عملية بالكامل: لا تُقرأ فقط، بل تُنفَّذ خطوة بخطوة على جهازك. كل مهمة تحتوي على: هدف واضح، أوامر جاهزة للنسخ، ومكاناً مخصصاً تلصق فيه مخرجات الأوامر (نسمّيها الأدلة)، ثم قائمة فحص قبل التسليم. عند إكمال كل المهام ستملك حساباً جاهزاً على GitHub ومستودعاً يعمل على جهازك — وهما أساس كل الجلسات التالية.

EN

This worksheet is **hands-on**: you do every step on your own machine. Each task gives you a clear objective, copy-ready commands, and a space to paste your command output as **evidence**, followed by a checklist. By the end you will have a working GitHub account and a local repository — the foundation for every later session.

أهداف التعلّم / Learning objectives

- التمييز بوضوح بين **Git** (أداة على جهازك) و**GitHub** (خدمة على الإنترنت).
- تثبيت Git والتحقق من نجاح التثبيت وضبط الهوية (الاسم والبريد) بشكل صحيح.
- إنشاء حساب GitHub مهني وتأمينه بالتحقق بخطوتين ومفتاح SSH أو رمز وصول.
- فهم المناطق الثلاث: مجلد العمل ← منطقة الانتظار ← المستودع، وعلاقتها بـ GitHub.
- إنشاء أول مستودع شخصي وتنفيذ أول `commit` وأول `push` بنجاح.
- قراءة حالة المستودع بالأوامر `git status` و `git log` وربطها بالواجهة الرسومية.

قاعدتان قبل أن نبدأ / Two ground rules

1. اكتب الأوامر بيدك، لا تنسخها فقط. الخطأ في الأمر وإصلاحه هو أسرع طريقة للتعلّم.
2. لا تخف من الخطأ. كل مستودع محلي نسخة مستقلة، وأي تجربة خاطئة يمكن التراجع عنها لاحقاً بأمر واحد — وستتعلّم هذه الأوامر في ورقة العمل 04.

الجزء الأول · المفاهيم الأساسية / Part 1 · Core concepts

ما هو Git؟ وما هو GitHub؟

Git برنامج مجاني مفتوح المصدر يعمل على جهازك، يتتبع كل تعديل تجريه على ملفات المشروع، ويحفظ التاريخ بشكل نقاط تحقّق (Commits) يمكن الرجوع إليها في أي وقت. أما **GitHub** فهو موقع إلكتروني يستضيف مستودعات Git على الإنترنت، ويضيف فوقها طبقة تعاون: طلبات السحب (Pull Requests)، المراجعة (Review)، المشكلات (Issues)، والصلاحيات.

EN

Git is a free, open-source tool that runs on your computer. It tracks every change you make and stores history as checkpoints called commits. **GitHub** is a website that hosts Git repositories online and adds a collaboration layer on top: pull requests, code review, issues, and permissions.

Git وGitHub؟ ما الفرق بين

Git is a tool on your computer · GitHub is a service on the internet

Git — برنامج على جهازك

- يعمل بدون إنترنت وبدون حساب
- يتتبع التعديلات ويسجل الالتزامات
- ينشئ الفروع ويعيد الدمج ويحلّ التعارضات
- git يحفظ التاريخ كاملاً في مجلد

local tool · no account · no internet needed

GitHub — خدمة على الإنترنت

- يحتاج حساباً واتصالاً بالإنترنت
- يستضيف المستودعات ويعمل نسخاً احتياطياً
- طلبات السحب والمراجعة والتعليقات
- المشكلات، الإجراءات، ونشر الصفحات

online service · account required · teamwork

هو المكان الذي نجتمع فيه — ولا يعمل أحدهما بدل الآخر GitHub هو المحرك، و Git: العلاقة

شكل 1: Git أداة محلية، وGitHub خدمة سحابية للتعاون — local tool vs. online service

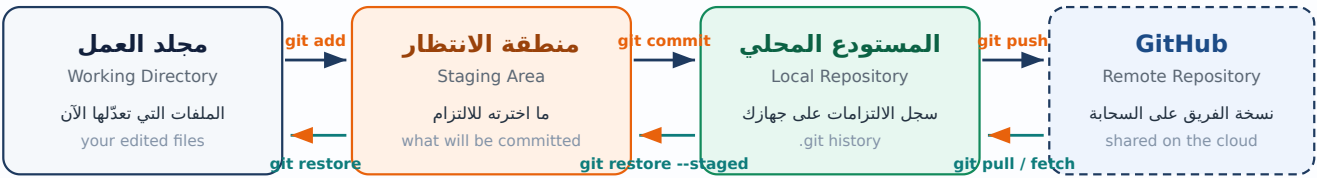
دليل سريع: إذا كان الأمر يبدأ بـ `git` فهو يعمل على جهازك؛ وإذا كان يتطلب فتح الموقع أو يتضمن كلمة `origin` فهو يتعامل مع GitHub. وبدون Git مثبتاً على جهازك لن تستطيع التعامل مع GitHub من سطر الأوامر إطلاقاً.

EN Quick test: a command that starts with `git` works on your machine; anything that needs the website or mentions `origin` talks to GitHub. Without Git installed locally you cannot work with GitHub from the command line at all.

المناطق الثلاث في Git

كل ملف تمرّ عليه ينتقل بين ثلاث مناطق. فهم هذه الصورة يوفّر عليك ساعات من الحيرة لاحقاً، لأن 90% من رسائل Git الغامضة تعني ببساطة: «هذا الملف في منطقة، وأنت تحاول تنفيذ أمر يخص منطقة أخرى».

EN Every file moves through three areas. Understanding this picture saves hours of confusion later, because most cryptic Git messages simply mean: “this file is in one area, and you are running a command that belongs to another.”



القاعدة: لكل التزام رسالة واضحة، ولكل تعديل كبير فرع خاص.

Rule: every commit needs a clear message; every big change gets its own branch.

شكل 2: من مجلد العمل إلى GitHub عبر منطقة الانتظار — working directory → staging → local repo → remote

الأمر / Command	المجموعة التي يعمل عليها	التشبيه العملي
<code>git status</code>	لا يعدّل شيئاً — يعرض الحالة	لوحة القيادة
<code>git add</code>	ينقل من مجلد العمل إلى منطقة الانتظار	تحضير الطلب

الأمر / Command	المنطقة التي يعمل عليها	التشبيه العملي
<code>git commit</code>	يحفظ من منطقة الانتظار إلى المستودع المحلي	تسليم الطلب
<code>git push</code>	يرفع من المستودع المحلي إلى GitHub	إرسال الطلب للفروع
<code>git pull</code>	يجلب من GitHub إلى جهازك	استلام تحديثات الزملاء

انتباه Watch out

`git push` وحده لا يرفع أي ملف لم تلتزمه بـ `git commit` ، و `git commit` وحده لا يرفع شيئاً إلى GitHub. الترتيب الإلزامي دائماً: `add` ← `commit` ← `push` .

مصطلحاتك الأولى Your first glossary

المصطلح	النطق التقريبي	المعنى بالعربية	English gloss
Repository	ريبازتوري	مستودع: مجلد مشروع يتتبعه Git	A tracked project folder
Commit	كوميّت	التزام: نقطة تحقّق محفوظة برسالة	A saved checkpoint with a message
Branch	برانش	فرع: مسار عمل مستقل عن <code>main</code>	An independent line of work
Main	مين	الفرع الرئيسي المستقر للمشروع	The stable primary branch
Remote	ريموت	مستودع بعيد على GitHub	A repository hosted online
Clone	كلون	نسخ المستودع كاملاً إلى جهازك	Copy a repo to your machine
Push / Pull	پوش / پُل	رفع تعديلاتك / جلب تعديلات الزملاء	Upload / download changes
Pull Request	پُل ريكوست	طلب سحب: اقتراح دمج فرعك بعد المراجعة	A merge proposal awaiting review
Merge	ميرج	دمج تعديلات فرع داخل فرع آخر	Combine branches
Conflict	كونفليكّت	تعارض: تعديلات متضاربان على السطر نفسه	Competing edits on the same line
Staging	ستينجنج	منطقة الانتظار قبل الالتزام	The pre-commit area
Issue	إيشو	مشكلة أو مهمة مسجّلة في المستودع	A tracked task or bug

الجزء الثاني · المهام العملية Part 2 · Hands-on tasks

1 تثبيت Git والتحقق منه

Install Git and verify it

Terminal Linux macOS Windows 10 نقطة ★ مبتدئ 15 دقيقة ★

الهدف: أن يتعرّف الجهاز على الأمر `git` ويُظهر رقم الإصدار.

(1) التثبيت حسب نظامك:

النظام	الطريقة الأسهل	ملاحظة مهمة
Windows	حمّل من <code>git-scm.com</code> ثم التثبيت بالإعدادات الافتراضية	سيُضاف Git Bash — استخدمه بدل موجه الأوامر
macOS	نقّذ <code>xcode-select --install</code> أو <code>brew install git</code>	الطريقة العادية تكفي

النظام	الطريقة الأسهل	ملاحظة مهمة
Linux (Debian/Ubuntu)	<code>sudo apt update && sudo apt install git</code>	الطريقة العادية تكفي

2) افتح الطرفية (Git Bash في Windows) وتحقق من التثبيت:

```
verify / التحقق - Terminal
$ git --version
git version 2.39.5
```

ملاحظة Note

رقم الإصدار عندك قد يختلف (2.4x أو أحدث) — المهم ألا تظهر رسالة `command not found` أو `'git' is not recognized`.

3) افحص مكان التثبيت (اختياري لكن مفيد عند حل المشكلات):

```
Terminal - Windows / macOS / Linux
# Windows (Git Bash)
$ which git
/mingw64/bin/git

# macOS / Linux
$ which git
/usr/bin/git
```

دليل المطلوب لهذه المهمة / Evidence for this task

الصق مخرجات الأمر `git --version` كما ظهرت على جهازك:

```
$ git --version
```

اذكر نظام التشغيل الذي تستخدمه:

خطأ شائع Common mistake

إذا رأيت `command not found` فأغلق النافذة وافتح واحدة جديدة بعد التثبيت، ثم أعد المحاولة. في Windows تأكد أنك تستخدم **Git Bash** لا `cmd`.

2 ضبط هوية Git (الاسم والبريد)

Configure your Git identity

git config

★ 10 نقطة

◆ مبتدئ

★ 10 دقائق

الهدف: أن تُوقع كل التزاماتك باسمك وبريدك الصحيحين، لأن هذه البيانات تظهر للفريق في سجل المشروع ولا يمكن تصحيحها لاحقاً بسهولة.

```
$ git config --global user.name "سارة الحسن"
$ git config --global user.email "sara@example.com"
$ git config --global init.defaultBranch main
$ git config --global pull.rebase true
$ git config --global core.editor "code --wait"
```

تحذير مهم جداً Critical

استخدم نفس البريد المسجل في حساب GitHub؛ فإن اختلف البريد فلن يرتبط التزامك بحسابك على GitHub، وقد يظهر باسم غريب بلا صورة ولا إحصاءات.

تحقق من الإعدادات ثم اختبر قراءتها:

```
$ git config --list
user.name=سارة الحسن
user.email=sara@example.com
init.defaultbranch=main
pull.rebase=true
core.editor=code --wait

$ git config user.name
سارة الحسن
```

دليل المطلوب Evidence

الصق مخرجات `git config --list`:

```
$ git config --list
```

3 إنشاء حساب GitHub مهني

Create a professional GitHub account

Browser

المتصفح

★ 15 نقطة

◆ مبتدئ

★ 15 دقيقة

الهدف: حساب جاهز للعمل الجماعي، باسم مستخدم واضح وتأمين جيد.

الخطوات:

1. افتح github.com واضغط **Sign up**.
2. أدخل البريد الإلكتروني (المدرسة أو الشخصي الموثوق) وكلمة مرور قوية (12 حرفاً على الأقل: حروف كبيرة وصغيرة وأرقام ورمز).
3. اختر اسم المستخدم بعناية - سيبقى معك في المشروع والمراجعات:

اسم جيد ✓	اسم ضعيف ✗	السبب
sara-alhassan	sara2009xxx	أرقام عشوائية توحى بعدم الجدية
omar-qa-dev	pro_gamer_99	لا يوصف الدور المهني
leen-haddad	asdfgh	لا يُقرأ ولا يُتذكَّر

4. أكمل التحقق من البريد واضغط الرابط في رسالة GitHub.

5. **فعل التحقق بخطوتين (2FA):** الصورة الشخصية ← Settings ← Password and authentication ← Enable Two-factor authentication (استخدم تطبيق مصادقة مثل Microsoft Authenticator أو Google Authenticator). **احفظ رموز الاستعادة في مكان آمن** — بدونها قد تفقد حسابك.
6. **حسن ملفك الشخصي:** صورة أو رمز واضح، اسم حقيقي، ووصف مختصر (مثال: طالب هندسة - أتعلم تطوير الويب). هذه أول ما يراه المراجع في طلب سحبك.

لماذا 2FA إلزامية؟ Why 2FA is mandatory

من 2023 صار GitHub يفرض التحقق بخطوتين على من ينشر على مستودعات عامة. في هذه الورشة كل الطلاب ينشرون على مستودع مشترك، لذا 2FA شرط تسليم لا اختيار.

دليل المطلوب Evidence

- اسم المستخدم الذي اخترته:
- رابط ملفك الشخصي (مثال: <https://github.com/sara-alhassan>):
- ضع علامة على ما أتممت: ☐ تأكيد البريد ☐ تفعيل 2FA ☐ صورة ووصف

4 تأمين الاتصال: مفتاح SSH أو رمز وصول

Secure your connection: SSH key or token

GitHub

ssh-keygen

★ 15 نقطة

◆ متوسط

★ 15 دقيقة

الهدف: تمكين الجهاز من الرفع إلى GitHub دون كتابة كلمة المرور كل مرة، وبأمان.

لماذا لا تكفي كلمة المرور؟ GitHub لم يعد يقبل كلمة مرور الحساب في أوامر git. أمامك طريقان: **رمز الوصول الشخصي (PAT)** وهو أسهل، أو **مفتاح SSH** وهو الأفضل لجهازك الشخصي لأنه لا ينتهي صلاحيته.

EN

Why not the password? GitHub no longer accepts account passwords for git commands. You have two options: a **Personal Access Token (PAT)** — easiest — or an **SSH key**, which is better on a personal machine because it does not expire.

الطريق أ — رمز الوصول الشخصي (الأسرع):

GitHub | إنشاء PAT

- 1) Settings ← Developer settings ← Personal access tokens ← Tokens (classic)
- 2) Generate new token (classic)
- 3) Note: workshop-laptop Expiration: 30 days
- 4) Scopes: اختر read:user (أقل صلاحية ممكنة) و repo
- 5) Generate token ← انسخ الرمز فوراً واحفظه في مدير كلمات المرور

تحذير أمني Security warning

الرمز يظهر مرة واحدة فقط. لا تضعه في ملف داخل المستودع، ولا في محادثة، ولا في صورة شاشة. عند طلب كلمة المرور في git، الصق الرمز بدل كلمة المرور.

الطريق ب — مفتاح SSH (الموصى به):

```
$ ssh-keygen -t ed25519 -C "sara@example.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sara/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Your identification has been saved in /home/sara/.ssh/id_ed25519
Your public key has been saved in /home/sara/.ssh/id_ed25519.pub

$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... sara@example.com

$ ssh -T git@github.com
Hi sara-alhassan! You've successfully authenticated, but GitHub does not provide shell access.
```

انشر المفتاح: GitHub ← Settings ← SSH and GPG keys ← New SSH key ← الصق سطر ... ssh-ed25519 ... كاملاً ← Add SSH key .

دليل المطلوب / Evidence

- الطريق الذي اخترته: ☐ SSH ☐ PAT
- الناتج المتوقع عند نجاح الاتصال (SSH): `Hi <username>! You've successfully authenticated...`
- الصق هنا مخرجات `ssh -T git@github.com` أو تأكيد إنشاء PAT:

```
$ ssh -T git@github.com
```

5 تجهيز بيئة العمل: VS Code والطرفية

Set up your workspace: VS Code and the terminal

VS Code

★ 10 نقطة

◆ مبتدئ

★ 10 دقائق

الهدف: بيئة تحرير مريحة تُظهر لك حالة Git بلمحة بصر.

1. نزل Visual Studio Code وثبته.
2. من داخل VS Code افتح **Extensions** وثبتت (Arabic Language Pack اختياري):

الإضافة	الفائدة
Arabic Language Pack	واجهة عربية إن رغبت
GitLens	يظهر لك من عدّل كل سطر ومق (مهم في المراجعة)
Prettier	تنسيق تلقائي للملفات
Live Server	تشغيل ملف HTML مباشرة في المتصفح

3. افتح الطرفية المدمجة: `Terminal ← New Terminal` (اختصار: `Ctrl+``).

4. تحقّق أن الطرفية تعرف git وضبط اتجاه أوامرك:

```
$ git --version
git version 2.39.5
$ git config user.name
سارة الحسن
```

5. جولة سريعة على أوامر الطرفية التي ستستخدمها في كل جلسة:

الأمر	الوظيفة	مثال
<code>pwd</code>	أين أنا الآن؟	<code>/c/Users/sara/projects</code>
<code>ls</code>	ماذا يوجد هنا؟	<code>ls -la</code>
<code>cd</code>	الانتقال إلى مجلد	<code>cd class-team-hub</code>
<code>cd ..</code>	الرجوع خطوة للخلف	
<code>mkdir</code>	إنشاء مجلد جديد	<code>mkdir projects</code>
<code>clear</code>	تنظيف الشاشة	
<code>history</code>	الأوامر السابقة	

دليل المطلوب / Evidence

الصق مخرجات `pwd` و `ls` من مجلد مشاريعك:

```
$ pwd
```

قائمة الإضافات التي تثبتها:

6 أول مستودع شخصي: clone ثم commit ثم push

Your first repository: clone, commit, push

git push

git commit

git clone

★ 25 نقطة

متوسط

★ 20 دقيقة

الهدف: تجربة الدورة الكاملة بنفسك على مستودع تجريبي شخصي (لم تُسَلَّم بعد المستودع المشترك – سيبدأ في ورقة العمل 02).

الخطوة 1 – أنشئ مستودعاً فارغاً على GitHub:

1. من أعلى الصفحة: + ← New repository .

2. الاسم: <اسم المستخدم>-git-practice – مثال: git-practice-sara-alhassan .

3. الوصف: ورشة الصف – Git تديب شخصي على أوامر .

4. النوع: Public، وفعل Add a README file .

5. اضغط Create repository .

الخطوة 2 – انسخ المستودع إلى جهازك (clone):

```
$ cd ~/projects
$ git clone https://github.com/sara-alhassan/git-practice-sara-alhassan.git
Cloning into 'git-practice-sara-alhassan'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
Unpacking objects: 100% (3/3), done.

$ cd git-practice-sara-alhassan
$ ls
README.md
```

ماذا حدث؟ *What just happened?*

أمر `git clone` أنشأ مجلداً على جهازك يحتوي على كل ملفات المستودع + مجلد مخفي اسمه `.git`. فيه كامل التاريخ والإعدادات. لا تحذف أو تعدّل هذا المجلد يدوياً أبداً.

الخطوة 3 – عدّل ملف `README.md` داخل VS Code: أضف سطرين عن نفسك.

الخطوة 4 – افحص ثم التزم ثم ارفع:

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
       modified:   README.md

$ git diff
diff --git a/README.md b/README.md
--- a/README.md
+++ b/README.md
@@ -1,3 @@
 # git-practice-sara-alhassan
+GitHub وGit طالبة في ورشة+.
+أتعلم كيف يعمل الفريق على مشروع واحد+.

$ git add README.md
$ git commit -m "docs(readme): add a short personal introduction"
[main 3f1a9c2] docs(readme): add a short personal introduction
1 file changed, 2 insertions(+)

$ git push
Enumerating objects: 5, done.
To https://github.com/sara-alhassan/git-practice-sara-alhassan.git
9d4b1a7..3f1a9c2  main → main
```

الخطوة 5 – افحص التاريخ وتأكد أن التعديل ظهر على GitHub:

```
$ git log --oneline
3f1a9c2 (HEAD → main, origin/main) docs(readme): add a short personal introduction
9d4b1a7 Initial commit

$ git remote -v
origin https://github.com/sara-alhassan/git-practice-sara-alhassan.git (fetch)
origin https://github.com/sara-alhassan/git-practice-sara-alhassan.git (push)
```

دليل المطلوب / Evidence

- رابط مستودعك: `https://github.com/...../.....`
- الصق مخرجات `git log --oneline` (سطران على الأقل):

```
$ git log --oneline
```

- الصق مخرجات `git status` في اللحظة الأخيرة (يجب أن تكون نظيفة):

```
$ git status
```

معيار الإتمام / Definition of done

تكون المهمة مكتملة عندما ترى على صفحة مستودعك في GitHub آخر التزام باسمك ورسالتك، من دون أي رسالة خطأ في الطرفية.

الجزء الثالث · تحقق ومراجعة / Part 3 · Check your understanding

أسئلة تحقق سريعة (15 نقطة)

س1. ما الفرق بين `git add` و `git commit` ؟ اذكر ماذا يحدث في كل خطوة ولماذا نحتاجهما معاً.

إجابتك / Your answer

س2. لو رفعت تعديلاً إلى GitHub ثم فتح زميلك المتصفح ولم يرَ أي تغيير، فما الاحتمالان الأكثر شيوعاً؟

إجابتك / Your answer

س3. اكتب الأمر الذي يُظهر: (أ) حالة المشروع الحالية، (ب) آخر الالتزامات في سطر واحد لكل التزام.

إجابتك / Your answer

س4. لماذا نستخدم اسم مستخدم مهنيًا وصورة واضحة في GitHub ؟ اذكر سببين مرتبطين بالعمل الجماعي.

إجابتك / Your answer

س5. ما الذي يحدث إذا استخدمت بريداً مختلفاً في `git config user.email` عن بريد حسابك على GitHub؟

إجابتك / Your answer

قبل التسليم Before you submit

قائمة الفحص النهائية / Final checklist

- ☐ [] تُنْذَر `git --version` بنجاح وأظهر رقماً لا خطأ (المهمة 1)
- ☐ [] صُبط `user.name` و `user.email` بنفس بريد GitHub (المهمة 2)
- ☐ [] الحساب مؤكَّد والتحقق بخطوتين مُفعَّل، والملف الشخصي مكتمل (المهمة 3)
- ☐ [] PAT أو مفتاح SSH يعمل واختُبر فعلاً (المهمة 4)
- ☐ [] VS Code مثبت مع GitLens، والطرفية المدمجة تعمل (المهمة 5)
- ☐ [] مستودع شخصي أنشئ ووُصل إليه بـ `clone` ورفُع إليه `commit` (المهمة 6)
- ☐ [] جميع الأدلة أعلاه مملوءة بمخرجات حقيقية لا منسوخة من الورقة

قيم نفسك بصدق قبل التسليم / Rate yourself honestly

4
أتقنت

3
أنجزت

2
بدأت

1
لم أبدأ

بطاقة التسليم / Submission card

البند	ما يُسَلَّم
رابط الحساب	<code>https://github.com/.....</code>
رابط المستودع الشخصي	<code>https://github.com/...../.....</code>
طريقة التعريف	PAT ☐ SSH ☐
اسم نظام التشغيل	
الوقت الذي استغرقته فعلاً	

ما القادم؟ / What's next

في ورقة العمل 02 ستُنسخ مستودع الفريق `class-team-hub` ، وتفهم بنيته، وتشغّل موقع الفريق على جهازك، وتجري تعديلك الأول على فرع خاص بك.